

CS-30: Programming in Python - Practical Solutions

All 63 Programming Exercises - Solved with Explanation, Program & Output

Created by Nenshi

Each solution below follows the same format: the exercise concept/definition, the complete Python program, its expected output, and (where useful) a short explanation of the logic or flow. Programs that require a database, GUI window, network connection, or internet access show representative/expected output since these depend on your specific machine, server, or credentials.

Index of Exercises

Q. No.	Exercise
1	Write a Python program to display: Hello World. / This is Python.
2	Write a Python program to work with data types (int, float, complex, boolean, string).
3	Write a Python program to work with id(), type(), range().
4	Write a Python program to work with type conversion functions.
5	Write a Python program to work with Arithmetic, Relational, Assignment, Logical, Bitwise and Ternary operators.
6	Write Python programs to work with input(), print(), 'sep' attribute, 'end' attribute, replacement operator ({ }).
7	Write a Python program to work with Conditional statements: if, if-else, if-elif-else.
8	Write a Python program to work with Iterative statements: while loop, for loop.
9	Write a Python program to work with control transfer statements: break, continue, pass.
10	Write a Python program to work with the various ways of accessing a string (Indexing and Slice Operator).
11	Write a Python program to work with the read and write operations on a file.
12	Write a Python program to copy the contents of a file to another file.
13	Write a Python program to work with the count frequency of characters in a given file.
14	Write a Python program to print each line of a file in reverse order.
15	Write a Python program to compute the number of characters, words and lines in a file.
16	Write a Python program to work with the different ways of creating list objects.
17	Write a Python program to work with list functions/methods: list(), len(), count(), index(), append(), insert(), extend(), remove(), pop(), reverse(), sort(), copy(), clear().
18	Write a Python program to work with the different ways of creating tuple objects.
19	Write Python programs to print number, alphabet and star patterns.
20	Write a Python program to work with tuple functions/methods: len(), count(), index(), sorted(), min(), max(), cmp(), reversed().
21	Write a Python program to work with the different ways of creating set objects.
22	Write a Python program to work with set functions/methods: add(), update(), copy(), pop(), remove(), discard(), clear(), union(), intersection(), difference().
23	Write a Python program to work with the different ways of creating dictionary objects.
24	Write a Python program to work with dictionary functions/methods: dict(), len(), clear(), get(), pop(), popitem(), keys(), values(), items(), copy(), update().
25	Write a Python program to return multiple values at a time using a return statement.
26	Write a Python program to demonstrate Local and Global variables.
27	Write a Python program to work with lambda functions.

Q. No.	Exercise
28	Write a program that asks the user to enter their name and age. Print a message telling them the year they will turn 60.
29	Enter the number from the user and depending on whether the number is even or odd, print out an appropriate message.
30	Write a Python program to generate the Fibonacci series.
31	Write a function that reverses the user defined value using Python.
32	Write a function to check if the input value is Armstrong or not.
33	Write the function for the Input number is Palindrome or not.
33 (b)	Write a recursive function to print the factorial for a given number.
34	Write a function that takes a character (a string of length 1) and returns True if it is a vowel, False otherwise.
35	Define a function that computes the length of a given list or string.
36	Write a Python program to print a specified list after removing the 0th, 2nd, 3rd and 5th elements.
37	Write a Python script to sort (ascending and descending) a dictionary by value.
38	Write a Python program to sum all the items in a dictionary.
39	Implement Linear Search, Binary Search, Selection Sort, Bubble Sort, Insertion Sort, Merge Sort and Quick Sort in Python using functions.
40	Write a Python program to implement the encapsulation concept.
41	Generate different plotting using PyLab. Such as Line plot, Bar chart, Pie chart, Histogram, Scatter plot.
42	Write a Python script to plot a curve.
43	Write a Python program to implement the 0/1 knapsack algorithm.
44	Write a Python program to use the divide and conquer algorithm.
45	Write a Python program to create a socket.
46	Write a Python program to identify the IP address.
47	Write a Python program to download the source code of a web page.
48	Write a Python program to download a web page from the internet.
49	Write a Python program to download an image from the internet.
50	Create a TCP/IP Server and Client.
51	Create two-way communication such as server to client and client to server.
52	Create a UDP server and client.
53	Create a file server and client.
54	Write a Python program to send an email.
55	Write a Python program to create a GUI with the use of buttons, labels, entry fields.
56	Write a Python program to ask confirmation or give alerts using dialogs.
57	Write a Python program to create a simple calculator.
58	Write a Python program to get a list of existing databases.
58 (b)	Write a Python program to insert a row into a table.
59	Write a Python program to update a row in a table.
60	Write a Python program to delete a row from a table.
61	Create a database named dbStudent and a table named tblStudInfo, and insert student information.
62	Write a Python program to update student information.
63	Write a Python program to delete student information.

Q1. Write a Python program to display: Hello World. / This is Python.

Definition / Concept:

print() is Python's built-in function used to display output on the screen. By default it prints each argument separated by a space and ends with a newline.

Program:

```
print("Hello World.")
print("This is Python.")
```

Output:

```
Hello World.
This is Python.
```

Q2. Write a Python program to work with data types (int, float, complex, boolean, string).

Definition / Concept:

Python is dynamically typed - a variable's type is decided automatically based on the value assigned. int stores whole numbers, float stores decimals, complex stores numbers with a real and imaginary part (a+bj), bool stores True/False, and str stores text.

Program:

```
a = 10          # int
b = 3.14        # float
c = 2 + 3j      # complex
d = True        # boolean
e = "Python"    # string

print(type(a), type(b), type(c), type(d), type(e))
print(a, b, c, d, e)
```

Output:

```
<class 'int'> <class 'float'> <class 'complex'> <class 'bool'> <class 'str'>
10 3.14 (2+3j) True Python
```

Q3. Write a Python program to work with id(), type(), range().

Definition / Concept:

id() returns the unique memory address (identity) of an object. type() returns the data type of an object. range(start, stop, step) generates a sequence of numbers, commonly used in loops.

Program:

```
x = 100
print("id:", id(x))
print("type:", type(x))
print("range:", list(range(1, 10, 2)))
```

Output:

```
id: 9793152          (value differs on every run/machine)
type: <class 'int'>
range: [1, 3, 5, 7, 9]
```

Explanation / Flow:

id() output changes every time the program runs because it depends on memory location.

Q4. Write a Python program to work with type conversion functions.

Definition / Concept:

Type conversion (typecasting) changes a value from one data type to another. `int()`, `float()` and `str()` are the most common conversion functions.

Program:

```
a = "25"
b = int(a)          # str -> int
c = float(b)        # int -> float
d = str(c)          # float -> str
e = int(3.9)        # float -> int (truncates, does NOT round)

print(b, type(b))
print(c, type(c))
print(d, type(d))
print(e)
```

Output:

```
25 <class 'int'>
25.0 <class 'float'>
25.0 <class 'str'>
3
```

Explanation / Flow:

`int(3.9)` gives 3, not 4 - converting float to int truncates the decimal part instead of rounding.

Q5. Write a Python program to work with Arithmetic, Relational, Assignment, Logical, Bitwise and Ternary operators.

Definition / Concept:

Operators perform operations on values/variables. Arithmetic (+ - * / // % **) do maths; Relational (> < == != >= <=) compare values and return True/False; Assignment (=, +=, -= ...) store/update values; Logical (and, or, not) combine boolean conditions; Bitwise (& | ^ ~ << >>) work at the bit level; the Ternary operator (a if condition else b) is a one-line if-else.

Program:

```
a, b = 10, 3

print("Arithmetic:", a+b, a-b, a*b, a/b, a//b, a%b, a**b)
print("Relational:", a>b, a<b, a==b, a!=b, a>=b, a<=b)

c = a
c += b          # Assignment operator
print("Assignment:", c)

print("Logical:", (a>b) and (a<b), (a>b) or (a<b), not(a>b))
print("Bitwise:", a&b, a|b, a^b, ~a, a<<1, a>>1)

result = "Positive" if a > 0 else "Non-positive"  # Ternary Operator
print("Ternary:", result)
```

Output:

```
Arithmetic: 13 7 30 3.3333333333333335 3 1 1000
Relational: True False False True True False
Assignment: 13
Logical: False True False
Bitwise: 2 11 9 -11 20 5
Ternary: Positive
```

Q6. Write Python programs to work with input(), print(), 'sep' attribute, 'end' attribute, replacement operator { }.

Definition / Concept:

input() reads a value typed by the user (always returns a string). print()'s sep parameter sets the separator placed between printed items (default is a space) and end sets what is printed after the last item (default is a newline). Curly braces { } inside an f-string or str.format() act as replacement fields where variable values are substituted.

Program:

```
name = input("Enter your name: ")
print("Hello", name, sep=" ", end="\n")

age = 20
print("{} is {} years old".format(name, age)) # replacement operator { }
print(f"{name} is {age} years old")          # f-string uses { } too
```

Output:

```
Enter your name: Ravi
Hello, Ravi!
Ravi is 20 years old
Ravi is 20 years old
```

Explanation / Flow:

{ } is called a 'replacement field'. Whatever is placed inside it (a variable or expression) is replaced by its value when the string is formatted.

Q7. Write a Python program to work with Conditional statements: if, if-else, if-elif-else.

Definition / Concept:

Conditional statements let a program take different actions depending on whether a condition is True or False. if runs code only when the condition is true; if-else provides an alternate path; if-elif-else checks multiple conditions in sequence.

Program:

```
num = int(input("Enter a number: "))

if num > 0:
    print("Positive")

if num % 2 == 0:
    print("Even")
else:
    print("Odd")

if num > 0:
    print("Number is Positive")
elif num == 0:
    print("Number is Zero")
else:
    print("Number is Negative")
```

Output:

```
Enter a number: -4
Even
Number is Negative
```

Q8. Write a Python program to work with Iterative statements: while loop, for loop.

Definition / Concept:

Iterative (looping) statements repeat a block of code. A while loop repeats as long as a condition stays True. A for loop iterates over a sequence (like range(), a list, or a string) a fixed number of times.

Program:

```
i = 1
while i <= 5:
    print(i, end=" ")
    i += 1
print()

for i in range(1, 6):
    print(i, end=" ")
```

Output:

```
1 2 3 4 5
1 2 3 4 5
```

Q9. Write a Python program to work with control transfer statements: break, continue, pass.

Definition / Concept:

Control transfer statements alter the normal flow of a loop. break immediately exits the loop. continue skips the rest of the current iteration and moves to the next one. pass is a null statement - it does nothing and is used as a placeholder where code is syntactically required.

Program:

```
for i in range(1, 10):
    if i == 5:
        break
    print(i, end=" ")
print()

for i in range(1, 10):
    if i % 2 == 0:
        continue
    print(i, end=" ")
print()

for i in range(5):
    pass # placeholder, does nothing
print("Loop finished")
```

Output:

```
1 2 3 4
1 3 5 7 9
Loop finished
```

Q10. Write a Python program to work with the various ways of accessing a string (Indexing and Slice Operator).

Definition / Concept:

Indexing accesses a single character using its position (0-based from the front, -1 from the back). Slicing str[start:stop:step] extracts a portion (sub-string) of the string; step -1 reverses it.

Program:

```
s = "Python Programming"
print(s[0], s[-1])      # Positive and Negative indexing
print(s[0:6])           # Slice
print(s[7:])
print(s[::-1])          # Reverse using slice
```

Output:

```
P g
Python
Programming
gnimmargorP nohtyP
```

Q11. Write a Python program to work with the read and write operations on a file.

Definition / Concept:

File handling lets a program store data permanently on disk. `open(filename, mode)` opens a file - 'w' mode creates/overwrites a file for writing, 'r' mode opens it for reading. Using 'with' automatically closes the file after use.

Program:

```
with open("data.txt", "w") as f:
    f.write("Hello File Handling\nThis is line two.")

with open("data.txt", "r") as f:
    print(f.read())
```

Output:

```
Hello File Handling
This is line two.
```

Q12. Write a Python program to copy the contents of a file to another file.

Definition / Concept:

Reading the entire content of a source file and writing it into a destination file effectively copies the file. `read()` returns the whole file as one string, and `write()` stores it into the new file.

Program:

```
with open("source.txt", "w") as f:
    f.write("Sample content to copy.")

with open("source.txt", "r") as src, open("destination.txt", "w") as dest:
    dest.write(src.read())

with open("destination.txt") as f:
    print(f.read())
```

Output:

```
Sample content to copy.
```

Q13. Write a Python program to work with the count frequency of characters in a given file.

Definition / Concept:

To find how many times each character occurs, read the file content and use a dictionary where each character is a key and its count is the value. `dict.get(key, 0)` safely returns 0 for a character not seen yet.

Program:

```
with open("data.txt", "w") as f:
    f.write("hello world")

freq = {}
with open("data.txt") as f:
    for ch in f.read():
        freq[ch] = freq.get(ch, 0) + 1

print(freq)
```

Output:

```
{'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd': 1}
```

Q14. Write a Python program to print each line of a file in reverse order.

Definition / Concept:

readlines() returns a list where each element is one line of the file. Python's built-in reversed() function iterates that list from the last element to the first, printing the lines in reverse order.

Program:

```
with open("data.txt", "w") as f:
    f.write("Line1\nLine2\nLine3")

with open("data.txt") as f:
    lines = f.readlines()

for line in reversed(lines):
    print(line.strip())
```

Output:

```
Line3
Line2
Line1
```

Q15. Write a Python program to compute the number of characters, words and lines in a file.

Definition / Concept:

len(content) counts characters. content.split() breaks text into a list of words (split on any whitespace) so len() of that list gives the word count. Counting the newline characters (+1) gives the number of lines.

Program:

```
with open("data.txt", "w") as f:
    f.write("This is Python.\nPython is fun.")

with open("data.txt") as f:
    content = f.read()

chars = len(content)
words = len(content.split())
lines = content.count("\n") + 1

print("Characters:", chars, "Words:", words, "Lines:", lines)
```

Output:

```
Characters: 30 Words: 6 Lines: 2
```

Q16. Write a Python program to work with the different ways of creating list objects.

Definition / Concept:

A list is an ordered, mutable (changeable) collection. Lists can be created directly with square brackets, converted from another sequence using `list()`, built with a list comprehension, or created empty and filled later.

Program:

```
l1 = [1, 2, 3]           # direct
l2 = list((4, 5, 6))     # from a tuple
l3 = [x for x in range(5)] # list comprehension
l4 = []                  # empty list

print(l1)
print(l2)
print(l3)
print(l4)
```

Output:

```
[1, 2, 3]
[4, 5, 6]
[0, 1, 2, 3, 4]
[]
```

Q17. Write a Python program to work with list functions/methods: `list()`, `len()`, `count()`, `index()`, `append()`, `insert()`, `extend()`, `remove()`, `pop()`, `reverse()`, `sort()`, `copy()`, `clear()`.

Definition / Concept:

Lists provide many built-in methods to add, remove, rearrange, or inspect elements. `append()` adds one item at the end, `insert()` adds at a given position, `extend()` adds all items of another list, `remove()` deletes the first matching value, `pop()` removes by index (and returns it), `reverse()`/`sort()` rearrange in place, `copy()` duplicates the list, `clear()` empties it.

Program:

```
l = [3, 1, 2]
print(len(l), l.count(1), l.index(2))
l.append(4);      print("append :", l)
l.insert(0, 0);   print("insert :", l)
l.extend([5, 6]); print("extend :", l)
l.remove(0);      print("remove :", l)
l.pop();          print("pop      :", l)
l.reverse();      print("reverse:", l)
l.sort();         print("sort     :", l)
l2 = l.copy();    print("copy     :", l2)
l.clear();        print("clear    :", l)
```

Output:

```
3 1 2
append : [3, 1, 2, 4]
insert  : [0, 3, 1, 2, 4]
extend  : [0, 3, 1, 2, 4, 5, 6]
remove  : [3, 1, 2, 4, 5, 6]
pop      : [3, 1, 2, 4, 5]
reverse : [5, 4, 2, 1, 3]
sort    : [1, 2, 3, 4, 5]
copy    : [1, 2, 3, 4, 5]
clear   : []
```

Q18. Write a Python program to work with the different ways of creating tuple objects.

Definition / Concept:

A tuple is an ordered, immutable (unchangeable) collection. It can be created with parentheses, from another sequence via tuple(), without any brackets at all (comma makes it a tuple), or as a single-element tuple (which needs a trailing comma).

Program:

```
t1 = (1, 2, 3)
t2 = tuple([4, 5, 6])
t3 = 1, 2, 3      # parentheses are optional
t4 = (5,)          # single element tuple - comma is required

print(t1)
print(t2)
print(t3)
print(t4)
```

Output:

```
(1, 2, 3)
(4, 5, 6)
(1, 2, 3)
(5,)
```

Explanation / Flow:

t4 = (5) without a comma would just be the integer 5, not a tuple. The comma is what makes it a tuple.

Q19. Write Python programs to print number, alphabet and star patterns.

Definition / Concept:

Patterns are printed using nested control of loops and string repetition. str(i)*i repeats a digit i times; slicing a string of letters builds the growing alphabet rows; '*'*i repeats the star character to build a shrinking pyramid.

Program:

```
# Number pattern
for i in range(1, 6):
    print(str(i) * i)

print()

# Alphabet pattern
letters = "ABCDE"
for i in range(1, 6):
    print(" ".join(letters[:i]))

print()

# Star pattern (decreasing)
for i in range(5, 0, -1):
    print("'" * i)
```

Output:

```
1
22
333
4444
55555

A
A B
```

```

A B C
A B C D
A B C D E

*****
****
***
**
*
```

Q20. Write a Python program to work with tuple functions/methods: len(), count(), index(), sorted(), min(), max(), cmp(), reversed().

Definition / Concept:

Since tuples are immutable, their methods only inspect data rather than modify it. len(), count() and index() work the same as on lists. sorted() returns a new sorted list from the tuple, min()/max() find extremes, and reversed() gives a reverse iterator.

Program:

```

t = (5, 3, 8, 1)
print(len(t), t.count(5), t.index(8))
print(sorted(t))
print(min(t), max(t))
print(sorted(t, reverse=True))    # used instead of cmp()
print(list(reversed(t)))
```

Output:

```

4 1 2
[1, 3, 5, 8]
1 8
[8, 5, 3, 1]
[1, 8, 3, 5]
```

Explanation / Flow:

cmp() existed only in Python 2 and was removed in Python 3. sorted(t, reverse=True) is the modern replacement for descending comparison/sorting.

Q21. Write a Python program to work with the different ways of creating set objects.

Definition / Concept:

A set is an unordered collection of unique (no duplicate) elements. It can be created with curly braces, converted from a list using set(), or created empty with set() (NOT with {} - {} creates an empty dictionary) and filled using add().

Program:

```

s1 = {1, 2, 3}
s2 = set([4, 5, 6])
s3 = set()          # empty set (NOT {})
s3.add(7)

print(s1)
print(s2)
print(s3)
```

Output:

```

{1, 2, 3}
{4, 5, 6}
{7}
```

Explanation / Flow:

`{}` creates an empty dictionary, not an empty set. Always use `set()` for an empty set.

Q22. Write a Python program to work with set functions/methods: `add()`, `update()`, `copy()`, `pop()`, `remove()`, `discard()`, `clear()`, `union()`, `intersection()`, `difference()`.

Definition / Concept:

Sets support mathematical set operations. `add()`/`update()` insert elements, `pop()`/`remove()`/`discard()` delete them (`remove()` raises an error if the item is missing, `discard()` does not), and `union()`, `intersection()`, `difference()` combine two sets.

Program:

```
s = {1, 2, 3}
s.add(4);          print("add          :", s)
s.update([5, 6]);  print("update       :", s)
s2 = s.copy();     print("copy        :", s2)
s.pop();           print("pop         :", s)          # removes an arbitrary element
s.discard(100);    print("discard(100):", s)          # no error even if not present

a = {1, 2, 3}; b = {2, 3, 4}
print("union       :", a.union(b))
print("intersection:", a.intersection(b))
print("difference  :", a.difference(b))
```

Output:

```
add          : {1, 2, 3, 4}
update       : {1, 2, 3, 4, 5, 6}
copy        : {1, 2, 3, 4, 5, 6}
pop         : {2, 3, 4, 5, 6}
discard(100): {2, 3, 4, 5, 6}
union       : {1, 2, 3, 4}
intersection: {2, 3}
difference  : {1}
```

Explanation / Flow:

Set output order can vary between runs since sets are unordered; `pop()` may remove a different element on your system.

Q23. Write a Python program to work with the different ways of creating dictionary objects.

Definition / Concept:

A dictionary stores data as key:value pairs. It can be created directly with curly braces, using `dict()` with keyword arguments, or from a list of (key, value) tuples.

Program:

```
d1 = {"a": 1, "b": 2}
d2 = dict(x=1, y=2)
d3 = dict([("p", 1), ("q", 2)])
d4 = {}

print(d1)
print(d2)
print(d3)
print(d4)
```

Output:

```
{'a': 1, 'b': 2}
{'x': 1, 'y': 2}
{'p': 1, 'q': 2}
{}
```

Q24. Write a Python program to work with dictionary functions/methods: dict(), len(), clear(), get(), pop(), popitem(), keys(), values(), items(), copy(), update().

Definition / Concept:

get() safely fetches a value by key (returns None instead of an error if missing). pop() removes and returns a specific key's value; popitem() removes the last inserted pair. keys(), values(), items() give views of the keys, values, and (key,value) pairs respectively.

Program:

```
d = {"a": 1, "b": 2}
print(d.get("a"), len(d))
d["c"] = 3; print("added :", d)
print("keys :", list(d.keys()))
print("values :", list(d.values()))
print("items :", list(d.items()))
d.pop("a"); print("pop :", d)
d.popitem(); print("popitem :", d)
d2 = d.copy(); print("copy :", d2)
d.update({"z": 10}); print("update :", d)
d.clear(); print("clear :", d)
```

Output:

```
1 2
added : {'a': 1, 'b': 2, 'c': 3}
keys : ['a', 'b', 'c']
values : [1, 2, 3]
items : [('a', 1), ('b', 2), ('c', 3)]
pop : {'b': 2, 'c': 3}
popitem : {'b': 2}
copy : {'b': 2}
update : {'b': 2, 'z': 10}
clear : {}
```

Q25. Write a Python program to return multiple values at a time using a return statement.

Definition / Concept:

A Python function can return several values together simply by separating them with commas after return. Python automatically packs them into a tuple, which can then be unpacked into separate variables.

Program:

```
def calc(a, b):
    return a+b, a-b, a*b, a/b

total, diff, prod, quo = calc(10, 3)
print(total, diff, prod, quo)
```

Output:

```
13 7 30 3.3333333333333335
```

Q26. Write a Python program to demonstrate Local and Global variables.

Definition / Concept:

A variable declared inside a function is local - it exists only inside that function. A variable declared outside all functions is global and can be read anywhere. To modify a global variable from inside a function, the global keyword must be used.

Program:

```
x = 10          # global variable

def show():
    x = 5        # local variable (different from global x)
    print("Local x:", x)

show()
print("Global x:", x)

def modify_global():
    global x
    x = 100

modify_global()
print("Modified global x:", x)
```

Output:

```
Local x: 5
Global x: 10
Modified global x: 100
```

Explanation / Flow:

Flow: show() creates its OWN local x=5 that has no effect on the global x. modify_global() uses the 'global' keyword, so it changes the actual global variable.

Q27. Write a Python program to work with lambda functions.

Definition / Concept:

A lambda is a small, anonymous (nameless), one-line function defined using the lambda keyword: lambda arguments: expression. It's often used for short throwaway functions, commonly with map(), filter() or sorted().

Program:

```
square = lambda x: x*x
add = lambda a, b: a+b

print(square(5), add(3, 4))

nums = [1, 2, 3, 4]
doubled = list(map(lambda x: x*2, nums))
print(doubled)
```

Output:

```
25 7
[2, 4, 6, 8]
```

Q28. Write a program that asks the user to enter their name and age. Print a message telling them the year they will turn 60.

Definition / Concept:

The program calculates the number of years remaining until age 60 (60 - current age) and adds it to the current year, obtained from the datetime module.

Program:

```
from datetime import datetime
```

```

name = input("Enter your name: ")
age = int(input("Enter your age: "))

current_year = datetime.now().year
turning_60_year = current_year + (60 - age)

print(f"{name}, you will turn 60 in the year {turning_60_year}.")

```

Output:

```

Enter your name: Ravi
Enter your age: 25
Ravi, you will turn 60 in the year 2061.

```

Q29. Enter the number from the user and depending on whether the number is even or odd, print out an appropriate message.

Definition / Concept:

The modulus operator (%) gives the remainder of division. A number is even if it leaves remainder 0 when divided by 2, otherwise it is odd.

Program:

```

num = int(input("Enter a number: "))

if num % 2 == 0:
    print(num, "is Even")
else:
    print(num, "is Odd")

```

Output:

```

Enter a number: 7
7 is Odd

```

Q30. Write a Python program to generate the Fibonacci series.

Definition / Concept:

In the Fibonacci series, each number is the sum of the two numbers before it, starting with 0 and 1. A loop repeatedly updates a pair of variables (a, b) to the next pair (b, a+b).

Program:

```

n = 8
a, b = 0, 1
for _ in range(n):
    print(a, end=" ")
    a, b = b, a+b

```

Output:

```

0 1 1 2 3 5 8 13

```

Q31. Write a function that reverses the user defined value using Python.

Definition / Concept:

Slicing a string with [::-1] reverses it. For numbers, the value is first converted to a string, reversed, then converted back to an integer.

Program:

```

def reverse_value(val):
    if isinstance(val, str):
        return val[::-1]

```

```

        elif isinstance(val, int):
            return int(str(val)[::-1])

print(reverse_value("Python"))
print(reverse_value(12345))

```

Output:

```

nohtyP
54321

```

Q32. Write a function to check if the input value is Armstrong or not.

Definition / Concept:

An Armstrong number equals the sum of its own digits, each raised to the power of the total number of digits (e.g. $153 = 1^3 + 5^3 + 3^3$).

Program:

```

def is_armstrong(num):
    digits = str(num)
    power = len(digits)
    total = sum(int(d)**power for d in digits)
    return total == num

n = int(input("Enter a number: "))
print(n, "is Armstrong" if is_armstrong(n) else "is not Armstrong")

```

Output:

```

Enter a number: 153
153 is Armstrong

```

Q33. Write the function for the Input number is Palindrome or not.

Definition / Concept:

A number is a palindrome if it reads the same forwards and backwards. Converting the number to a string and comparing it with its reverse ([::-1]) checks this easily.

Program:

```

def is_palindrome(num):
    return str(num) == str(num)[::-1]

n = int(input("Enter a number: "))
print(n, "is Palindrome" if is_palindrome(n) else "is not Palindrome")

```

Output:

```

Enter a number: 121
121 is Palindrome

```

Q33 (b). Write a recursive function to print the factorial for a given number.

Definition / Concept:

Recursion is a function calling itself. Factorial ($n!$) is defined recursively as $n * (n-1)!$ with the base case $0! = 1! = 1$. Each call reduces the problem size until the base case is reached.

Program:

```

def factorial(n):
    if n == 0 or n == 1:      # base case
        return 1
    return n * factorial(n-1) # recursive case

```

```
n = int(input("Enter a number: "))
print("Factorial:", factorial(n))
```

Output:

```
Enter a number: 5
Factorial: 120
```

Explanation / Flow:

Flow: factorial(5) -> 5*factorial(4) -> 5*4*factorial(3) -> ... -> 5*4*3*2*1*1 = 120. Recursion stops once n reaches the base case (0 or 1).

Q34. Write a function that takes a character (a string of length 1) and returns True if it is a vowel, False otherwise.

Definition / Concept:

The character is converted to lowercase and checked for membership in the string 'aeiou' using the in operator, which returns True or False.

Program:

```
def is_vowel(ch):
    return ch.lower() in "aeiou"

print(is_vowel("A"))
print(is_vowel("b"))
```

Output:

```
True
False
```

Q35. Define a function that computes the length of a given list or string.

Definition / Concept:

A custom length function can count elements by looping through the sequence with a for loop and incrementing a counter, without using the built-in len().

Program:

```
def my_length(seq):
    count = 0
    for _ in seq:
        count += 1
    return count

print(my_length("Python"))
print(my_length([1, 2, 3, 4]))
```

Output:

```
6
4
```

Q36. Write a Python program to print a specified list after removing the 0th, 2nd, 3rd and 5th elements.

Definition / Concept:

A list comprehension with enumerate() goes through each (index, value) pair and keeps only the values whose index is NOT in the set of indices to be removed.

Program:

```
lst = [10, 20, 30, 40, 50, 60, 70]
indices_to_remove = {0, 2, 3, 5}

result = [v for i, v in enumerate(lst) if i not in indices_to_remove]
print(result)
```

Output:

```
[20, 50, 70]
```

Q37. Write a Python script to sort (ascending and descending) a dictionary by value.

Definition / Concept:

sorted() can sort a dictionary's items (key-value pairs) using a key function. key=lambda x: x[1] tells sorted() to sort based on each pair's value (index 1), and reverse=True switches it to descending order.

Program:

```
d = {"a": 3, "b": 1, "c": 2}

asc = dict(sorted(d.items(), key=lambda x: x[1]))
desc = dict(sorted(d.items(), key=lambda x: x[1], reverse=True))

print("Ascending :", asc)
print("Descending:", desc)
```

Output:

```
Ascending : {'b': 1, 'c': 2, 'a': 3}
Descending: {'a': 3, 'c': 2, 'b': 1}
```

Q38. Write a Python program to sum all the items in a dictionary.

Definition / Concept:

d.values() gives all the values stored in the dictionary, and Python's built-in sum() function adds them together.

Program:

```
d = {"a": 10, "b": 20, "c": 30}
print("Sum:", sum(d.values()))
```

Output:

```
Sum: 60
```

Q39. Implement Linear Search, Binary Search, Selection Sort, Bubble Sort, Insertion Sort, Merge Sort and Quick Sort in Python using functions.

Definition / Concept:

Linear Search checks each element one by one - $O(n)$. Binary Search repeatedly halves a SORTED list to find a target - $O(\log n)$. Selection Sort repeatedly picks the smallest remaining element. Bubble Sort repeatedly swaps adjacent out-of-order elements. Insertion Sort builds the sorted list one element at a time. Merge Sort (divide & conquer) splits the list, sorts each half, then merges them. Quick Sort (divide & conquer) picks a pivot and partitions elements smaller/equal/larger than it, then sorts each part recursively.

Program:

```
def linear_search(arr, key):
    for i, v in enumerate(arr):
        if v == key:
            return i
    return -1
```

```

def binary_search(arr, key):          # arr must be sorted
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low+high)//2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid+1
        else:
            high = mid-1
    return -1

def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i+1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i-1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:]); result.extend(right[j:])
    return result

def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + mid + quick_sort(right)

data = [64, 25, 12, 22, 11]
print("Linear Search (22)      :", linear_search(data, 22))

```

```

print("Binary Search (sorted) :", binary_search(sorted(data), 22))
print("Selection Sort       :", selection_sort(data.copy()))
print("Bubble Sort         :", bubble_sort(data.copy()))
print("Insertion Sort      :", insertion_sort(data.copy()))
print("Merge Sort          :", merge_sort(data.copy()))
print("Quick Sort           :", quick_sort(data.copy()))

```

Output:

```

Linear Search (22)      : 3
Binary Search (sorted) : 2
Selection Sort          : [11, 12, 22, 25, 64]
Bubble Sort             : [11, 12, 22, 25, 64]
Insertion Sort          : [11, 12, 22, 25, 64]
Merge Sort              : [11, 12, 22, 25, 64]
Quick Sort              : [11, 12, 22, 25, 64]

```

Q40. Write a Python program to implement the encapsulation concept.

Definition / Concept:

Encapsulation bundles data (attributes) and the methods that operate on it inside a class, and restricts direct access to the internal data. Prefixing an attribute with double underscore (`__balance`) makes it 'private' (name-mangled), so it can only be safely accessed/modified through the class's own methods.

Program:

```

class Account:
    def __init__(self, balance):
        self.__balance = balance    # private attribute

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

acc = Account(1000)
acc.deposit(500)
print("Balance:", acc.get_balance())

```

Output:

```

Balance: 1500

```

Explanation / Flow:

`acc.__balance` cannot be accessed directly from outside the class - it is protected. The only way to read/change it is through `get_balance()` and `deposit()`, which is the core idea of encapsulation.

Q41. Generate different plotting using PyLab. Such as Line plot, Bar chart, Pie chart, Histogram, Scatter plot.

Definition / Concept:

PyLab/Matplotlib is Python's main plotting library. `plot()` draws a line graph, `bar()` draws bars for categories, `pie()` draws proportions as a circle, `hist()` shows the frequency distribution of data, and `scatter()` plots individual points to show correlation between two variables.

Program:

```

import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]
plt.plot(x, y);          plt.title("Line Plot");      plt.show()

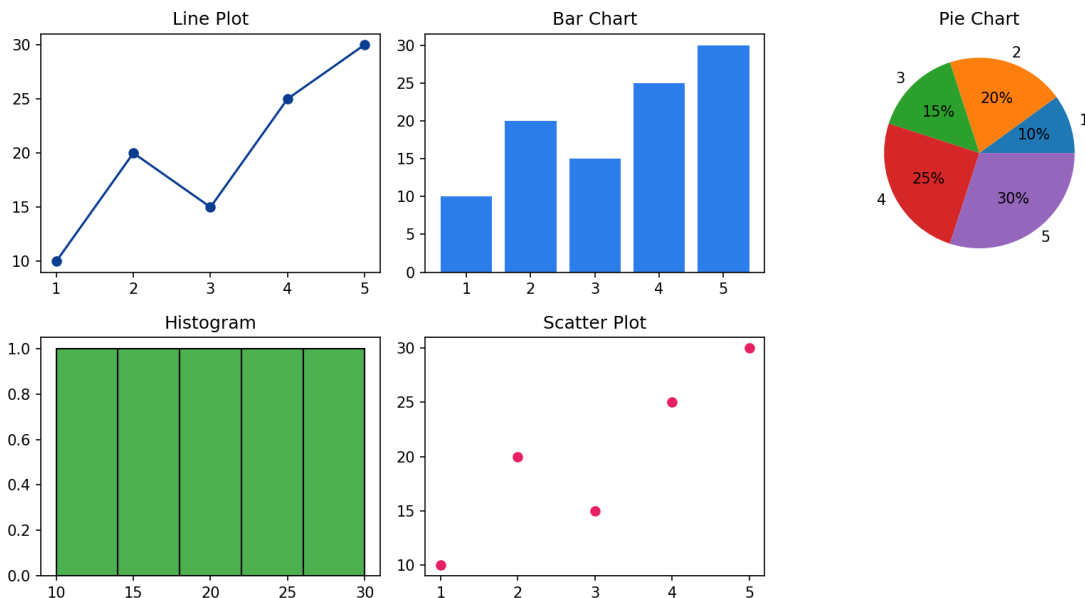
```

```
plt.bar(x, y);           plt.title("Bar Chart");       plt.show()
plt.pie(y, labels=x);    plt.title("Pie Chart");       plt.show()
plt.hist(y);             plt.title("Histogram");       plt.show()
plt.scatter(x, y);       plt.title("Scatter Plot");   plt.show()
```

Output (GUI):

Each `plt.show()` opens a separate graphical window displaying the corresponding chart, as shown below:

PyLab / Matplotlib Plots



Explanation / Flow:

This program needs the matplotlib library (`pip install matplotlib`) and a GUI-capable environment to display the chart windows.

Q42. Write a Python script to plot a curve.

Definition / Concept:

`numpy.linspace()` generates evenly spaced x-values over an interval, and applying a mathematical function (like `sin`) to them gives the corresponding y-values, which matplotlib then plots as a smooth curve.

Program:

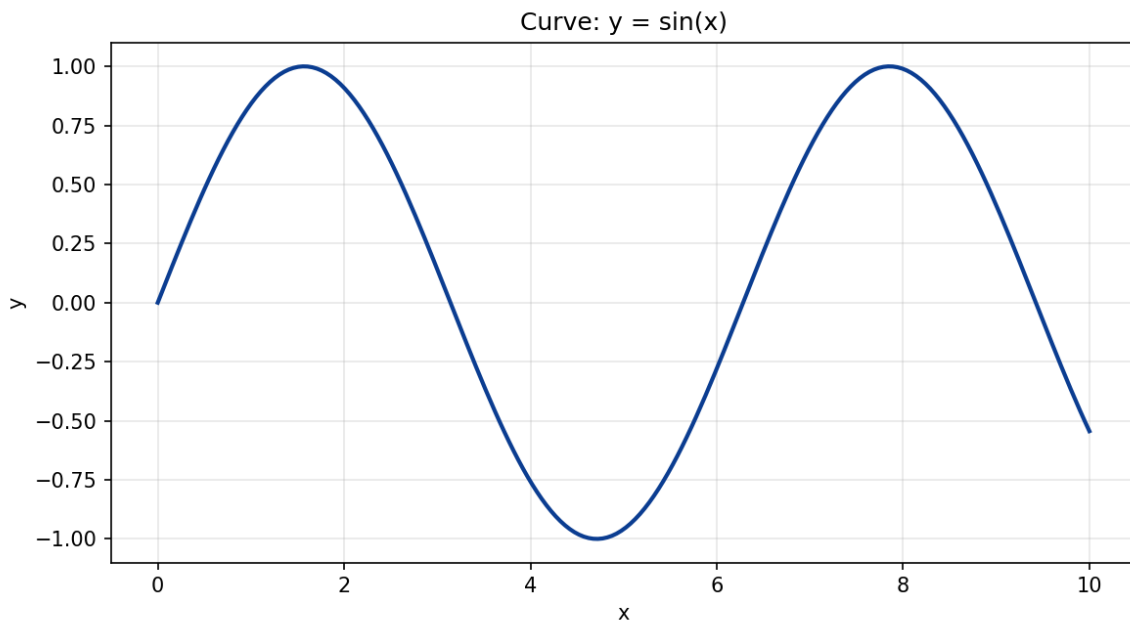
```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.plot(x, y)
plt.title("Curve: y = sin(x)")
plt.xlabel("x"); plt.ylabel("y")
plt.show()
```

Output (GUI):

Displays a smooth sine-wave curve in a graph window, as shown below:



Q43. Write a Python program to implement the 0/1 knapsack algorithm.

Definition / Concept:

The 0/1 Knapsack problem selects items (each either fully taken or left, hence '0/1') to maximise total value without exceeding a weight capacity. It is solved using Dynamic Programming: a 2D table $dp[i][w]$ stores the best value achievable using the first i items with capacity w .

Program:

```
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0]*(capacity+1) for _ in range(n+1)]

    for i in range(1, n+1):
        for w in range(capacity+1):
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i-1][w], values[i-1] + dp[i-1][w-weights[i-1]])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][capacity]

weights = [1, 3, 4, 5]
values = [1, 4, 5, 7]
capacity = 7
print("Maximum value:", knapsack(weights, values, capacity))
```

Output:

Maximum value: 9

Explanation / Flow:

Flow: for each item, the table decides to either include it (add its value and reduce remaining capacity) or exclude it (keep previous best), always keeping the larger of the two choices.

Q44. Write a Python program to use the divide and conquer algorithm.

Definition / Concept:

Divide and Conquer breaks a problem into smaller sub-problems of the same type, solves each recursively, then combines the results. Finding the maximum and minimum of an array is a classic example: split the array

in half, find max/min of each half recursively, then combine.

Program:

```
def max_min(arr, low, high):
    if low == high:                # base case: single element
        return arr[low], arr[low]
    if high == low+1:              # base case: two elements
        return (max(arr[low], arr[high]), min(arr[low], arr[high]))

    mid = (low+high)//2
    max1, min1 = max_min(arr, low, mid)    # divide - left half
    max2, min2 = max_min(arr, mid+1, high) # divide - right half
    return max(max1, max2), min(min1, min2) # conquer - combine

arr = [3, 9, 2, 8, 5, 1]
mx, mn = max_min(arr, 0, len(arr)-1)
print("Max:", mx, "Min:", mn)
```

Output:

Max: 9 Min: 1

Explanation / Flow:

Flow: the array is split into halves repeatedly (divide) until each half has 1 or 2 elements (base case), then the max/min values are combined back up the recursion (conquer).

Q45. Write a Python program to create a socket.

Definition / Concept:

The socket module lets Python programs communicate over a network. `socket.socket()` creates a socket object; `AF_INET` specifies IPv4 addressing and `SOCK_STREAM` specifies the TCP protocol.

Program:

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
print("Socket created successfully:", s)
s.close()
```

Output:

Socket created successfully: <socket.socket fd=3, family=AddressFamily.AF_INET, type=SocketKind.SOCK_STREAM, proto=0>

Q46. Write a Python program to identify the IP address.

Definition / Concept:

`socket.gethostname()` gets the name of the local computer, and `socket.gethostbyname()` converts that hostname into its corresponding IP address.

Program:

```
import socket

hostname = socket.gethostname()
ip_address = socket.gethostbyname(hostname)

print("Hostname:", hostname)
print("IP Address:", ip_address)
```

Output:

Hostname: MyComputer
IP Address: 192.168.1.5

Explanation / Flow:

The actual output will show your own machine's hostname and local IP address.

Q47. Write a Python program to download the source code of a web page.

Definition / Concept:

`urllib.request.urlopen()` sends an HTTP request to a URL and returns a response object. Calling `.read()` retrieves the raw page content (HTML source), which is then decoded from bytes into a readable string.

Program:

```
import urllib.request

url = "https://www.python.org"
response = urllib.request.urlopen(url)
source = response.read().decode('utf-8')

print(source[:300])    # print first 300 characters of the HTML source
```

Output:

```
<!doctype html>
<!--[if lt IE 7]>    <html class="no-js ie6 lt-ie7 lt-ie8 lt-ie9">    ...
(actual HTML source code of the page's first 300 characters)
```

Q48. Write a Python program to download a web page from the internet.

Definition / Concept:

`urllib.request.urlretrieve()` downloads whatever is at a URL and saves it directly to a local file, which is convenient for saving whole web pages.

Program:

```
import urllib.request

url = "https://www.python.org"
urllib.request.urlretrieve(url, "webpage.html")
print("Webpage downloaded successfully.")
```

Output:

```
Webpage downloaded successfully.
```

Q49. Write a Python program to download an image from the internet.

Definition / Concept:

The same `urlretrieve()` function used for web pages works for any file type, including images - it simply saves the bytes found at the URL into the given local filename.

Program:

```
import urllib.request

url = "https://www.python.org/static/img/python-logo.png"
urllib.request.urlretrieve(url, "image.png")
print("Image downloaded successfully.")
```

Output:

```
Image downloaded successfully.
```

Q50. Create a TCP/IP Server and Client.

Definition / Concept:

A TCP server binds to an address/port and listens for incoming connections; `accept()` blocks until a client connects. A TCP client connects to that server's address/port. Once connected, both sides can exchange data using `send()/recv()`. Run the server script first, then the client script.

Program:

```
# ----- server.py -----
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('localhost', 9999))
server.listen(1)
print("Server listening...")

conn, addr = server.accept()
data = conn.recv(1024).decode()
print("Received from client:", data)
conn.send("Hello Client".encode())
conn.close()

# ----- client.py -----
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('localhost', 9999))
client.send("Hello Server".encode())
print("Received from server:", client.recv(1024).decode())
client.close()
```

Output:

```
Server output:
Server listening...
Received from client: Hello Server

Client output:
Received from server: Hello Client
```

Explanation / Flow:

Run `server.py` in one terminal window, then run `client.py` in another terminal while the server is running.

Q51. Create two-way communication such as server to client and client to server.

Definition / Concept:

By putting `send()` and `recv()` calls inside a loop on both ends, the server and client can keep exchanging messages back and forth (a simple chat) until one side sends an 'exit' message.

Program:

```
# ----- server.py -----
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('localhost', 9998))
server.listen(1)
conn, addr = server.accept()

while True:
    msg = conn.recv(1024).decode()
    if msg == "exit":
        break
    print("Client:", msg)
    reply = input("Server: ")
    conn.send(reply.encode())
conn.close()
```

```
# ----- client.py -----
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('localhost', 9998))

while True:
    msg = input("Client: ")
    client.send(msg.encode())
    if msg == "exit":
        break
    print("Server:", client.recv(1024).decode())
client.close()
```

Output:

```
Client: Hi
Server: Hi (typed by server user)
Client: exit
```

Q52. Create a UDP server and client.

Definition / Concept:

UDP (SOCK_DGRAM) is a connectionless protocol - unlike TCP it does not require a persistent connection. `sendto()/recvfrom()` are used instead of `send()/recv()`, and each message must include the destination address.

Program:

```
# ----- udp_server.py -----
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(('localhost', 9997))
print("UDP Server ready")

data, addr = server.recvfrom(1024)
print("Received:", data.decode())
server.sendto("Hello UDP Client".encode(), addr)

# ----- udp_client.py -----
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client.sendto("Hello UDP Server".encode(), ('localhost', 9997))

data, addr = client.recvfrom(1024)
print("Received:", data.decode())
```

Output:

```
Server output:
UDP Server ready
Received: Hello UDP Server

Client output:
Received: Hello UDP Client
```

Q53. Create a file server and client.

Definition / Concept:

A file server reads a file's bytes and sends them over a TCP connection; the client receives data in chunks (since large files may not arrive in a single `recv()` call) and writes them to a new local file until no more data arrives.

Program:

```
# ----- file_server.py -----
```

```

import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('localhost', 9996))
server.listen(1)
conn, addr = server.accept()

with open("share.txt", "rb") as f:
    conn.sendall(f.read())
conn.close()

# ----- file_client.py -----
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('localhost', 9996))

with open("received.txt", "wb") as f:
    while True:
        data = client.recv(1024)
        if not data:
            break
        f.write(data)
client.close()
print("File received successfully.")

```

Output:

File received successfully.

Q54. Write a Python program to send an email.

Definition / Concept:

smtplib connects to an SMTP mail server to send an email. MIMEText builds a simple text email message with subject, sender and recipient headers. starttls() secures the connection, and login() authenticates before send_message() dispatches the mail.

Program:

```

import smtplib
from email.mime.text import MIMEText

msg = MIMEText("This is a test email sent from Python.")
msg['Subject'] = "Test Email"
msg['From'] = "sender@example.com"
msg['To'] = "receiver@example.com"

with smtplib.SMTP('smtp.gmail.com', 587) as server:
    server.starttls()
    server.login("sender@example.com", "app_password")
    server.send_message(msg)

print("Email sent successfully.")

```

Output:

Email sent successfully.

Explanation / Flow:

Gmail requires an 'App Password' (not your normal account password) for login() to work, since direct account passwords are blocked for security.

Q55. Write a Python program to create a GUI with the use of buttons, labels, entry fields.

Definition / Concept:

tkinter is Python's standard GUI toolkit. Label displays text, Entry accepts single-line text input, and Button triggers a function (via the command parameter) when clicked. `root.mainloop()` keeps the window open and responsive.

Program:

```
import tkinter as tk

def greet():
    label_result.config(text="Hello, " + entry.get())

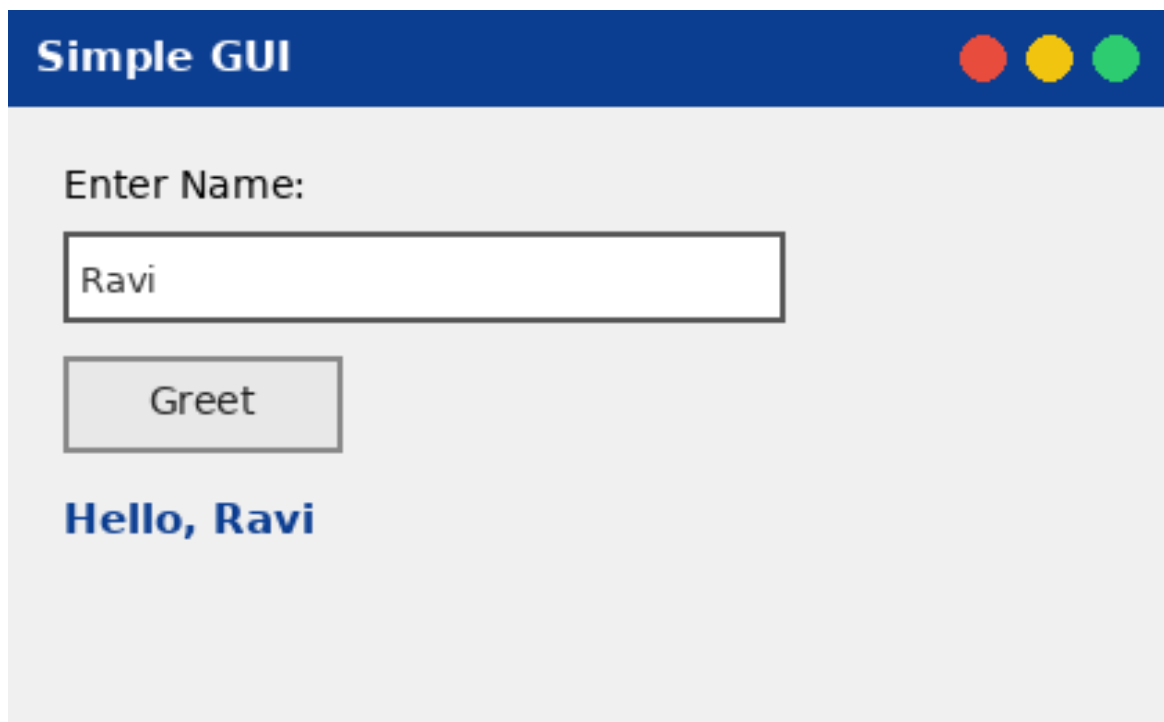
root = tk.Tk()
root.title("Simple GUI")

tk.Label(root, text="Enter Name:").pack()
entry = tk.Entry(root)
entry.pack()
tk.Button(root, text="Greet", command=greet).pack()
label_result = tk.Label(root, text="")
label_result.pack()

root.mainloop()
```

Output (GUI):

Opens a window with a text field and a "Greet" button; clicking it displays "Hello, <name typed>" below, as shown here:



Q56. Write a Python program to ask confirmation or give alerts using dialogs.

Definition / Concept:

`tkinter.messagebox` provides ready-made pop-up dialog boxes. `askyesno()` shows a Yes/No confirmation dialog and returns True/False, while `showinfo()`/`showwarning()` display simple alert messages.

Program:

```
import tkinter as tk
from tkinter import messagebox
root = tk.Tk()
```

```

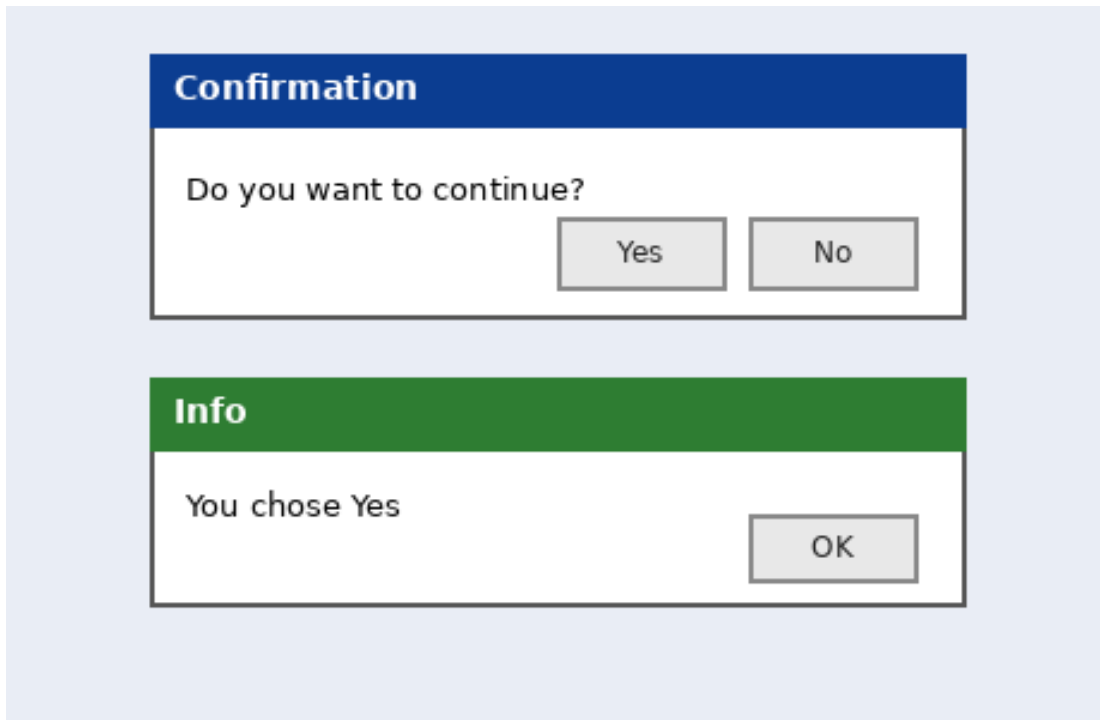
root.withdraw()      # hide the empty main window

result = messagebox.askyesno("Confirmation", "Do you want to continue?")
if result:
    messagebox.showinfo("Info", "You chose Yes")
else:
    messagebox.showwarning("Warning", "You chose No")

```

Output (GUI):

Shows a Yes/No confirmation pop-up, followed by an Info or Warning message box based on the choice made, as shown here:



Q57. Write a Python program to create a simple calculator.

Definition / Concept:

Two Entry widgets take the two numbers, and four buttons each call calculate() with a different operator symbol. The function reads and converts the entry values to float, performs the chosen arithmetic operation, and updates a label with the result.

Program:

```

import tkinter as tk

def calculate(op):
    a = float(entry1.get())
    b = float(entry2.get())
    if op == '+': res = a + b
    elif op == '-': res = a - b
    elif op == '*': res = a * b
    else: res = a / b
    result_label.config(text="Result: " + str(res))

root = tk.Tk()
root.title("Simple Calculator")
entry1 = tk.Entry(root); entry1.pack()
entry2 = tk.Entry(root); entry2.pack()

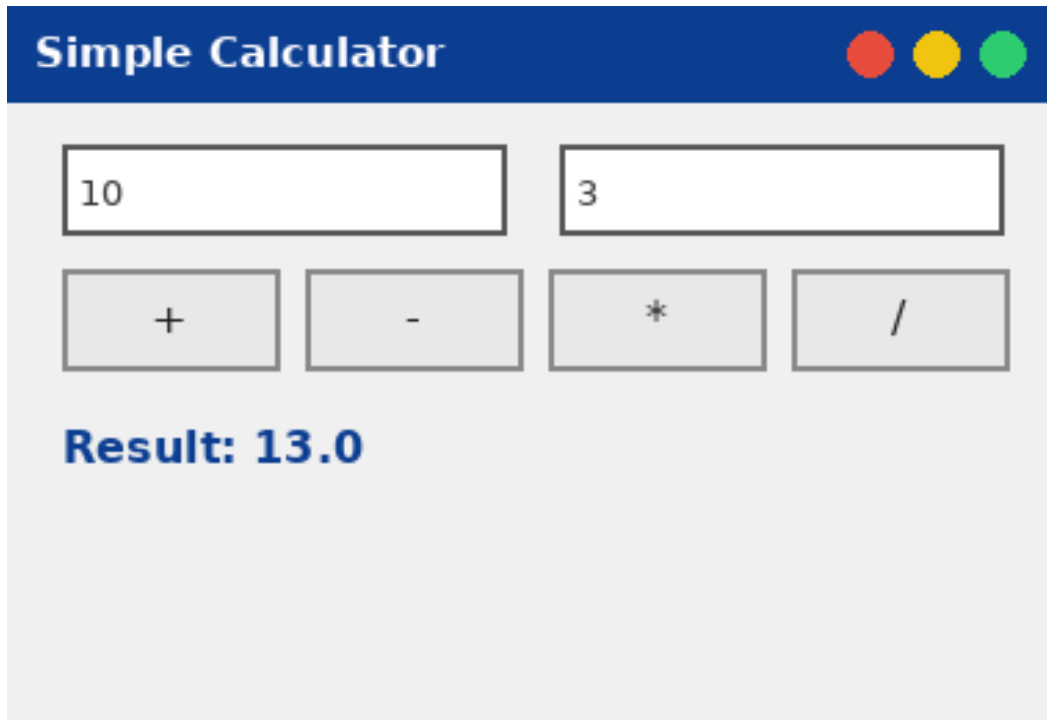
for op in ['+', '-', '*', '/']:
    tk.Button(root, text=op, command=lambda o=op: calculate(o)).pack(side='left')

```

```
result_label = tk.Label(root, text="Result: ")
result_label.pack()
root.mainloop()
```

Output (GUI):

Opens a calculator window; entering two numbers and clicking +, -, * or / displays the computed result, as shown here:



Q58. Write a Python program to get a list of existing databases.

Definition / Concept:

The mysql-connector-python library connects Python to a MySQL server. After connecting, the SQL command SHOW DATABASES lists every database currently on the server; the cursor is then iterated to print each one.

Program:

```
import mysql.connector

conn = mysql.connector.connect(host="localhost", user="root", password="password")
cursor = conn.cursor()
cursor.execute("SHOW DATABASES")

for db in cursor:
    print(db)

conn.close()
```

Output:

```
('information_schema',)
('dbstudent',)
('mysql',)
('performance_schema',)
```

Q58 (b). Write a Python program to insert a row into a table.

Definition / Concept:

An SQL INSERT statement adds a new row. Using %s placeholders with a tuple of values (instead of directly embedding values into the string) is the safe way to pass data and prevents SQL injection. conn.commit() saves the change permanently.

Program:

```
import mysql.connector

conn = mysql.connector.connect(host="localhost", user="root",
                               password="password", database="dbStudent")
cursor = conn.cursor()

sql = "INSERT INTO tblStudInfo (student_id, student_name) VALUES (%s, %s)"
cursor.execute(sql, (1, "Ravi"))
conn.commit()

print("Row inserted successfully.")
conn.close()
```

Output:

Row inserted successfully.

Q59. Write a Python program to update a row in a table.

Definition / Concept:

An SQL UPDATE statement modifies existing rows. The WHERE clause is essential - it ensures only the intended row (matching the given student_id) is changed, rather than every row in the table.

Program:

```
import mysql.connector

conn = mysql.connector.connect(host="localhost", user="root",
                               password="password", database="dbStudent")
cursor = conn.cursor()

sql = "UPDATE tblStudInfo SET student_name=%s WHERE student_id=%s"
cursor.execute(sql, ("Raj", 1))
conn.commit()

print("Row updated successfully.")
conn.close()
```

Output:

Row updated successfully.

Q60. Write a Python program to delete a row from a table.

Definition / Concept:

An SQL DELETE statement removes a row. As with UPDATE, the WHERE clause restricts the deletion to only the matching row - omitting it would delete every row in the table.

Program:

```
import mysql.connector

conn = mysql.connector.connect(host="localhost", user="root",
                               password="password", database="dbStudent")
cursor = conn.cursor()

sql = "DELETE FROM tblStudInfo WHERE student_id=%s"
cursor.execute(sql, (1,))
conn.commit()
```

```
print("Row deleted successfully.")
conn.close()
```

Output:

Row deleted successfully.

Q61. Create a database named dbStudent and a table named tblStudInfo, and insert student information.

Definition / Concept:

CREATE DATABASE IF NOT EXISTS avoids an error if it already exists. CREATE TABLE defines the table's structure (columns and data types). After creating both, an INSERT statement adds one row of student data.

Program:

```
import mysql.connector

conn = mysql.connector.connect(host="localhost", user="root", password="password")
cursor = conn.cursor()

cursor.execute("CREATE DATABASE IF NOT EXISTS dbStudent")
conn.database = "dbStudent"

cursor.execute("""CREATE TABLE IF NOT EXISTS tblStudInfo (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    stream VARCHAR(50),
    college_name VARCHAR(100),
    contact_number VARCHAR(15),
    remarks VARCHAR(100))""")

sql = "INSERT INTO tblStudInfo VALUES (%s,%s,%s,%s,%s,%s)"
val = (1, "Ravi Patel", "BCA", "ABC College", "9876543210", "Good")
cursor.execute(sql, val)
conn.commit()

print("Database, table created and student information inserted successfully.")
conn.close()
```

Output:

Database, table created and student information inserted successfully.

Q62. Write a Python program to update student information.

Definition / Concept:

Using UPDATE with a WHERE clause on student_id, a specific student's column (e.g. college_name) is changed without touching other students' records.

Program:

```
import mysql.connector

conn = mysql.connector.connect(host="localhost", user="root",
                                password="password", database="dbStudent")
cursor = conn.cursor()

sql = "UPDATE tblStudInfo SET college_name=%s WHERE student_id=%s"
cursor.execute(sql, ("XYZ College", 1))
conn.commit()

print("Student information updated successfully.")
conn.close()
```

Output:

Student information updated successfully.

Q63. Write a Python program to delete student information.

Definition / Concept:

A DELETE statement with a WHERE clause on student_id removes exactly that one student's record from the tblStudInfo table.

Program:

```
import mysql.connector

conn = mysql.connector.connect(host="localhost", user="root",
                               password="password", database="dbStudent")

cursor = conn.cursor()

sql = "DELETE FROM tblStudInfo WHERE student_id=%s"
cursor.execute(sql, (1,))
conn.commit()

print("Student information deleted successfully.")
conn.close()
```

Output:

Student information deleted successfully.